**Assignment No. 1**

**Title: Demonstrate How to Create GitHub Account**

**Aim:**

To create a GitHub account and understand the process of registering, verifying, and setting up for collaboration.

**Software Requirements:**

- Operating System: Windows 10 / 11
- Browser: Chrome / Edge / Firefox
- Internet Connection

**Theory:**

GitHub is a cloud-based platform used for version control and collaboration. It allows developers to host repositories, share code, and track project changes using Git. Creating a GitHub account is the first step toward working on version-controlled projects and contributing to open-source software.

**Procedure:**

1. Open a web browser and go to https://github.com.
2. Click on **"Sign up"** in the top-right corner.
3. Enter details:
    - Email address
    - Password
    - Username
    - Verify account using CAPTCHA
4. Choose account preferences (Free / Paid plan).
5. Click **"Continue"** or **"Complete setup."**
6. Open your email inbox and verify your account by clicking the **verification link** sent by GitHub.
7. Your GitHub account is now created successfully.

**Output:**

- Successfully created and verified GitHub account.
- Screenshot of GitHub profile page.

**Result:**

Thus, the GitHub account was created successfully and can now be used to host and manage repositories.

**Assignment No. 2**

**Title: Exploring Git Commands through Collaborative Coding**

**Aim:**

To study and execute basic Git commands for setting up a repository, creating branches, committing, and merging changes collaboratively.

**Software Requirements:**

- Operating System: Windows 10 / 11
- Software: Git (Installed)
- Internet Connection

**Theory:**

Git is a distributed version control system that allows multiple developers to work on a project simultaneously. It helps track changes, collaborate efficiently, and maintain version history.

**Procedure:**

**1. Setting Up Git Repository**

git config --global user.name "Vrushali Tambe"

git config --global user.email "vrusha08@gmail.com"

mkdir collaborative-project

cd collaborative-project

git init

**2. Creating and Committing Changes**

echo "Welcome" > index.txt

git add index.txt

git commit -m "Initial commit with index.txt"

git remote add origin https://github.com/vrusha8/collab-git-practical.git

git push origin master

**3. Branching and Merging**

git checkout -b feature-login

echo "Welcome to IMRD" > login.txt

git add login.txt

git commit -m "Add login functionality"

git checkout master

git merge feature-login

git push origin master

**Output:**

Successfully created repository, committed files, and merged branch.

**Result:**

Git repository operations like commit, branch, and merge were successfully performed.

**Assignment No. 3**

**Title: Implement GitHub Operations using Git**

**Aim:**

To implement Git operations such as cloning a repository, making changes, creating branches, and pushing/pulling updates using Git commands.

**Software Requirements:**

Git, Windows OS, Internet access, GitHub account.

**Procedure:**

**1. Cloning a Repository**

git clone https://github.com/vrusha8/collab-git-practical.git

**2. Making Changes and Creating a Branch**

git checkout -b feature-update

echo "This is a new feature" > feature.txt

git add feature.txt

git commit -m "Added feature.txt file"

**3. Push / Pull Changes to GitHub**

git push origin feature-update

git pull origin feature-update

**Output:**

Repository cloned and new branch successfully created and pushed to GitHub.

**Result:**

GitHub operations using Git were executed successfully.

**Assignment No. 4**

**Title: Create Docker Account**

**Aim:**

To create a Docker Hub account for managing Docker images and containers.

**Software Requirements:**

- OS: Windows 10 / 11
- Browser: Chrome / Edge

- Internet Connection

**Theory:**

Docker Hub is a cloud-based registry service for building, sharing, and managing container images. Creating an account allows developers to push and pull container images and automate workflows.

**Procedure:**

1. Visit https://hub.docker.com/signup
2. Fill in the form:
    - Username
    - Email address
    - Password
3. Click **"Sign Up."**
4. Open your email and verify your account.
5. Once verified, log in at https://hub.docker.com/login.

**Output:**

Docker Hub account created successfully.

**Result:**

A Docker account was successfully created and verified.

**Assignment No. 6**

**Title: Demonstrate a Practical on Version Control Tools**

**Aim:**

To perform basic version control operations using Git.

**Procedure:**

**Part 1 – Initialize Repository**

```
mkdir my-git-practical
cd my-git-practical
git init
echo "My First Git Repo" > index.txt
git add index.txt
git commit -m "Initial commit"
```

**Part 2 – Make and Track Changes**

```
echo "This is a Git practical session." >> index.txt
git status
```

git diff

git add index.txt

git commit -m "Updated index with session info"

git log

**Part 3 – Push to GitHub (Optional)**

git remote add origin https://github.com/vrusha08/my-website.git

git branch -M main

git push -u origin main

**Output:**

Repository initialized, changes committed, and version history displayed.

**Result:**

Version control operations were successfully performed using Git.

**Assignment No. 7**

**Title: Create and Review Merge Request on GitLab**

**Aim:**

To study and perform merge requests on GitLab and review the changes collaboratively.

**Procedure:**

1. Log in to your GitLab account → Create a new project.
2. Clone the repository:

- git clone https://gitlab.com/username/merge-request-demo.git
- cd merge-request-demo

3. Create a new branch and add content:

- git checkout -b feature-branch
- echo "My First Git Repo" > index.txt
- git add .
- git commit -m "Added index.txt"
- git push origin feature-branch

4. Go to GitLab → Click **Create Merge Request**.
5. Review as another user, add comments, approve, and merge.

**Output:**

Merge request created, reviewed, and merged successfully.

**Result:**

GitLab merge request and code review were demonstrated successfully.

**Assignment No. 8**

**Title: Build a Docker Image for a Sample Web Application Using Dockerfile**

**Aim:**

To study Dockerfile instructions and build an image for a sample web application.

**Procedure:**

1. Create project directory:

```
mkdir docker-webapp
cd docker-webapp
```

2. Create index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Docker Web App</title>
</head>
<body>
  <h1>Hello, Docker!</h1>
  <p>This is a sample web application running in Docker.</p>
</body>
</html>
```

3. Create Dockerfile:

```
FROM nginx
COPY index.html /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

4. Build and run the image:

```
docker build -t mywebapp .
docker run -d -p 8080:80 mywebapp
```

5. Open browser → http://localhost:8080

**Output:**

Displays “Hello, Docker!” page in the browser.

**Result:**

Docker image for a sample web application was successfully built and executed.